



Basics of Parallel Computing 2026S Assignment 2

May 27, 2026

2 Person Group 1

1: Ida NEUHAUSER, 11806595
2: Max SPANNRING, 12104980

1 Exercise 1

1.1 What do a and t count?

In the given loop, a records which thread executes at the current iteration, and t counts how many tasks a thread is assigned.

1.2 Values for all elements in a and t

case / a	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
static	0	0	0	0	0	1	1	1	1	1	2	2	2	2	2
static,2	0	0	1	1	2	2	0	0	1	1	2	2	0	0	1
static,3	0	0	0	1	1	1	2	2	2	0	0	0	1	1	1
dynamic,1	0	1	1	0	2	2	2	1	2	0	0	1	0	0	0
dynamic,4	1	1	1	1	2	2	2	2	1	1	1	1	0	0	0
guided,7	0	0	0	0	0	0	0	1	1	1	1	1	1	1	2

case / t	0	1	2
static	5	5	5
static,2	6	5	4
static,3	6	6	3
dynamic,1	7	4	4
dynamic,4	3	8	4
guided,7	7	7	1

2 Exercise 2

2.1 Optimal Schedule

Since the workload in total is 56, it is distributed equally across the cores, each has a workload of 14. Figure 1 shows a possible assignment, core 1 and core 2 do two larger tasks, core 3 gets assigned five medium sized tasks and core 4 does nine medium to smaller tasks.

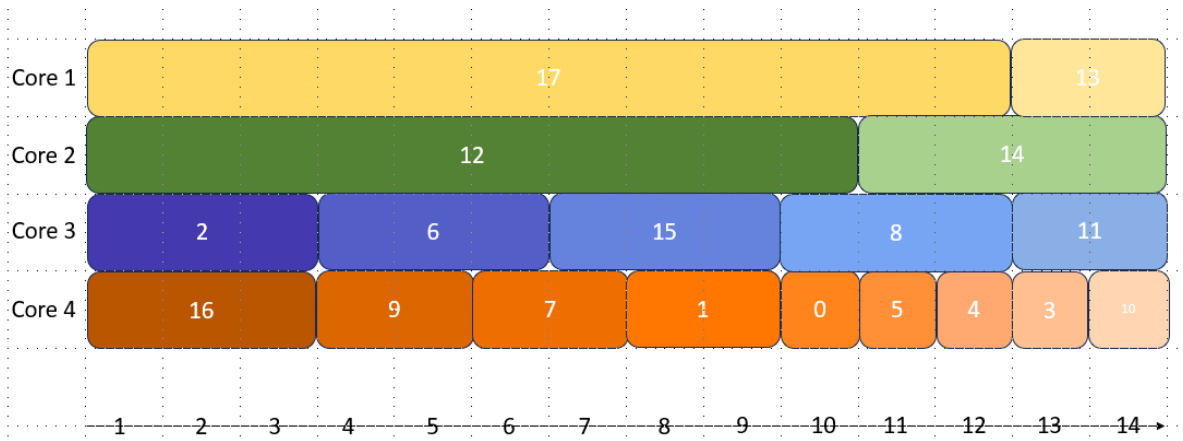
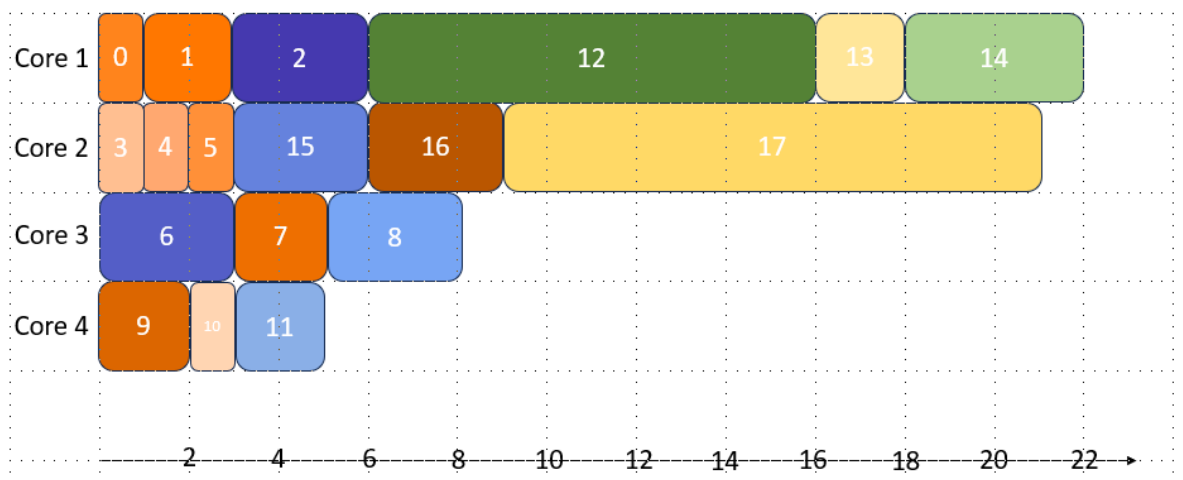


Figure 1: Schedule with optimal load balancing.

2.2 Schedule static,3

Using `static,3`, the workload is unbalanced. Core 1 and 2 get two packages, cores 3 and 4 only one (Figure 2). This leads to a makespan of 22.

Figure 2: Schedule with `static,3` assignment.

2.3 Schedule dynamic,2

For the `dynamic,2` schedule shown in Figure 3, the package was assigned to the earliest free core, which leads to a makespan of 21.

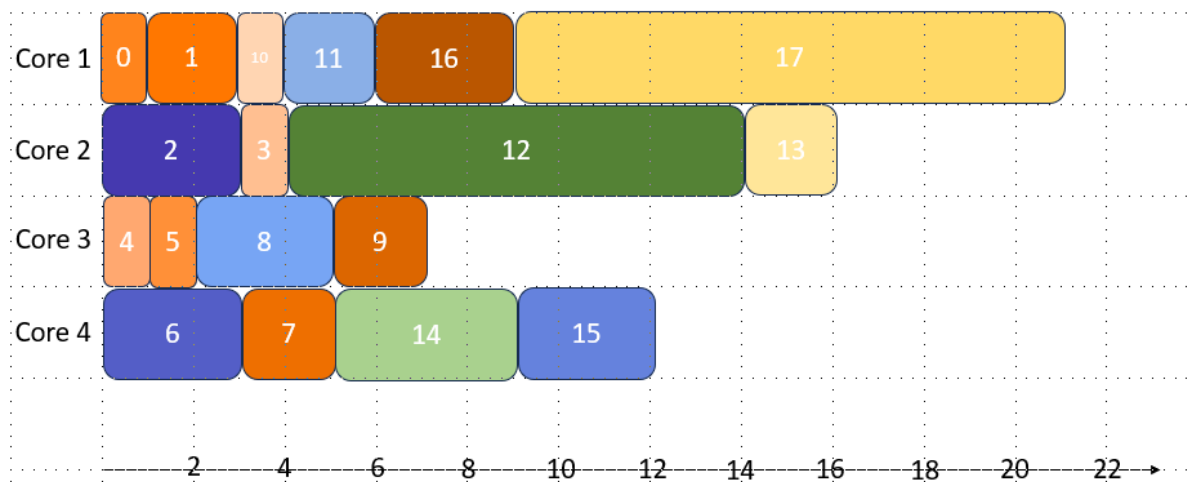


Figure 3: Schedule with dynamic, 2 assignment.

3 Exercise 3

3.1 What is the output of the three different versions?

The different `#pragma` functions only affect which thread executes what task, not the numerical computation. Therefore, `res = 5` for VERSION A, VERSION B and VERSION C.

- VERSION A: `res = 5`. There is no race because each task writes a different variable and `#pragma omp taskwait` ensures that the parent thread does not read `a` and `b` before the writing is finished.
- VERSION B: `res = 5`. Same as before, different tasks write to different variables and finish writing before they are read for return. One thread starts `res = omp_tasks(5)`, the other threads work on tasks from `#pragma omp task shared`.
- VERSION C: `res = 5`. Each thread executes `res = omp_tasks(5)` and overwrites the value, but the result is the same each time.

3.2 How often is the function `omp_tasks` called?

For each $v \leq 2$, `omp_tasks = 0T` is called once, for all larger v , it the number of calls depends on the two previous numbers.

`OT(5)` needs

`OT(4)` needs

`OT(3)` needs

`OT(2)`

`OT(1)`

`OT(2)`

`OT(3)` needs

OT(2)

OT(1)

- VERSION A: `omp_tasks` gets called $1 \cdot OT(5) + 1 \cdot OT(4) + 2 \cdot OT(3) + 3 \cdot OT(2) + 2 \cdot OT(1) = 9$ times.
- VERSION B: Same as in Version A, `omp_tasks` gets called $1 \cdot OT(5) + 1 \cdot OT(4) + 2 \cdot OT(3) + 3 \cdot OT(2) + 2 \cdot OT(1) = 9$ times. `single` `nowait` and `task shared (b)` only affect which thread executes what and does not change the number of executions.
- VERSION C: Because of `#pragma omp critical`, each thread executes `res = omp_tasks(5)`. Thus, `omp_tasks` gets called $3 \cdot 9 = 27$ times.

4 Exercise 6

4.1 Convert OpenMP code to CUDA

The `kernel_gpu.cu` file has been modified in order to compute Julia sets on the GPU. We replaced the empty stub with three pieces: a `__device__` copy of `checkPointForJuliaSet` (the per-pixel escape-time loop, body unchanged from `kernel_cpu.cpp`), a `__global__` kernel `julia_kernel_device` that derives its (x, y) from `blockIdx/blockDim/threadIdx` and computes one pixel per thread, and a host-side `julia_kernel` wrapper that picks a default 16×16 block size (or honors `-blocksize`), does `cudaMalloc` $\rightarrow$ kernel launch $\rightarrow$ `cudaMemcpy` back to the host buffer $\rightarrow$ `cudaFree`. The doubly-nested CPU loop disappears entirely — its iterations are now expressed as the grid of threads launched on the GPU, with a bounds check inside the kernel to handle the case where the grid slightly over-covers the image.

4.2 Running time analysis

In this task, we had to run the CUDA program on the GPU for different resolutions. The default block size of 16×16 and `nrep=5` was used.

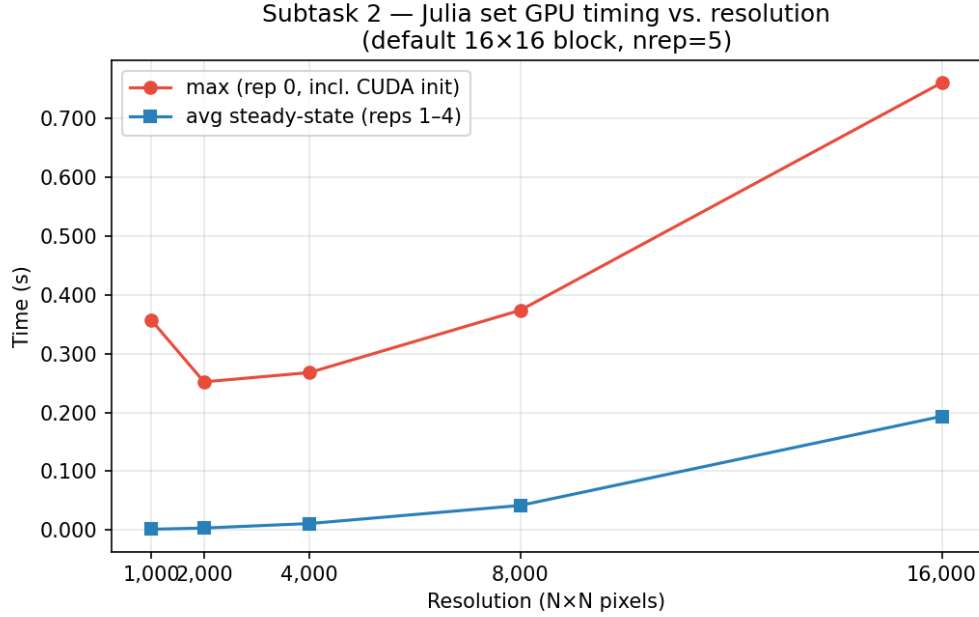


Figure 4: GPU kernel time vs. resolution (default 16×16 block size, nrep = 5). *Red*: maximum time across 5 reps (rep 0, includes CUDA context initialisation). *Blue*: average steady-state time (reps 1–4).

Table 1: Subtask 2: GPU timing vs. resolution. avg_steady_s excludes rep 0 (init-dominated).

Resolution	Avg steady (s)	Max (s)	Speedup vs. 1 000
1000	0.0012	0.3572	1.00
2000	0.0033	0.2521	2.61
4000	0.0109	0.2679	8.77
8000	0.0417	0.3739	33.49
16000	0.1934	0.7612	155.26

Rep 0 is always the max and is dominated by CUDA context initialization. The remaining reps scale closely to $O(N^2)$

4.3 Impact of block size

Now we fix the resolution to 2000×2000, and try out different block sizes. We can see that (1, 32) is the best block size. The output array is indexed as `julia_set[x*res_y + y]`, so adjacent values of `y` occupy adjacent memory locations. With block shape (1, 32), all 32 threads in a warp share the same `x` but have consecutive `y` values, mapping to consecutive memory addresses — the warp issues a single 128-byte L1 transaction (a fully coalesced write). With block shape (32, 1), by contrast, threads share the same `y` but have consecutive `x` values, so their target addresses lie `res_y` floats (= 80 000 bytes) apart. The warp must then issue 32

separate L1 transactions, one per thread (fully non-coalesced).

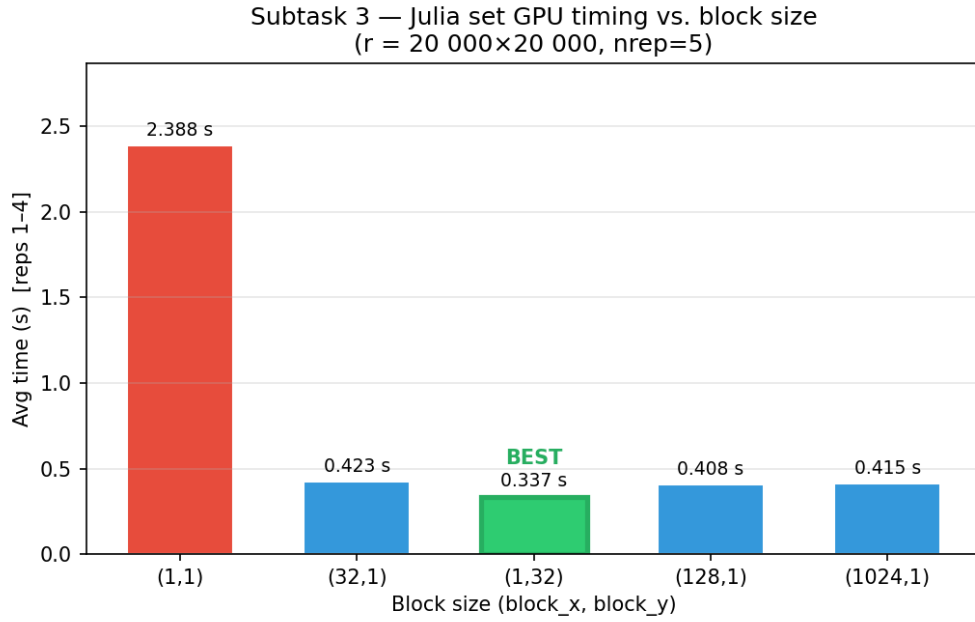


Figure 5: Average steady-state GPU time per block size ($r = 20\,000 \times 20\,000$, nrep = 5, reps 1–4). Best configuration (block 1×32) highlighted in green.

Table 2: Subtask 3: timing and ncu profiler metrics per block size ($r = 20\,000 \times 20\,000$). GST = global store throughput (L1 texture pipe). Warp eff = mean active threads per warp instruction (max 32).

Block	Threads	Avg (s)	Occupancy (%)	GST (GB/s)	Warp eff
1x1	1	2.388	48.6	2.16	1.0
32x1	32	0.423	49.03	48.03	23.73
1x32	32	0.337	48.97	6.37	23.74
128x1	128	0.408	81.07	59.85	23.73
1024x1	1024	0.415	63.02	41.82	23.74

4.4 Running time: CPU vs. GPU

For this subtask, we want to compare the running time of the code on the CPU vs the GPU. We can see that there is no crossover - the GPU wins in every measured case. However, we can see that the code runtime scales similarly for both the CPU and GPU version.

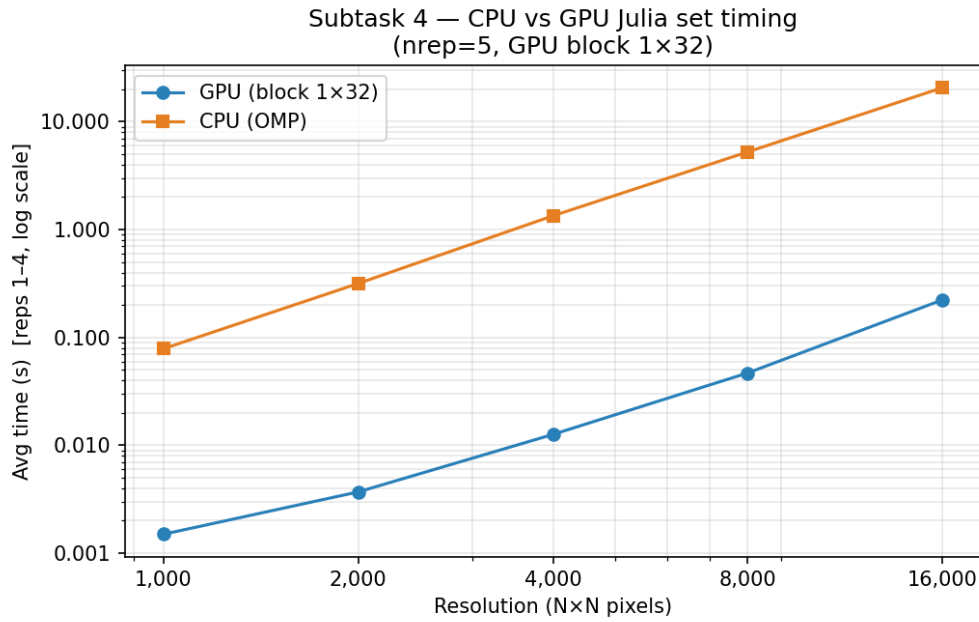


Figure 6: CPU vs. GPU average steady-state time vs. resolution (log–log scale, nrep = 5, reps 1–4). GPU uses best block (1×32); CPU uses OMP_NUM_THREADS=32. No crossover was detected in the measured resolution range.

Table 3: Subtask 4: CPU vs. GPU average steady-state time and speedup. Speedup = $t_{\text{CPU}}/t_{\text{GPU}}$.

Resolution	GPU avg (s)	CPU avg (s)	Speedup
1000	0.0015	0.0787	52.52
2000	0.0037	0.3166	85.60
4000	0.0126	1.3425	106.39
8000	0.0469	5.2497	112.00
16000	0.2229	20.6730	92.75